МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №3

з дисципліни **“Функціональне програмування”**

на тему **“НЕЗДАТНІ ДО ЗМІН СТРУКТУРИ ДАНИХ”**

Варіант 13. Страхові поліси

Виконав:
Студент групи ФЕП-23
Маньківський Максим Володимирович

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: зрозуміти та застосувати незмінні структури даних Python (`tuple`, `frozenset`, `@dataclass(frozen=True, slots=True)`, `MappingProxyType`), замінити мутації предметної моделі на повернення нових копій (pure updates) і виміряти ефект переходу на тестах.

Теоретичні відомості:

Функціональний стиль спирається на чисті функції та незмінні структури. Незмінність дає референтну прозорість, просту композицію, безпечне спільне читання і хешованість: незмінний об'єкт може бути ключем словника, елементом множини та аргументом `lru_cache`. У стандартній бібліотеці Python для цього є `tuple` замість `list`, `frozenset` замість `set`, `@dataclass(frozen=True)` для доменних сутностей (поля не можна присвоїти, усі «оновлення» — через `dataclasses.replace`, який повертає нову копію) і `MappingProxyType` — read-only представлення словника.

`MappingProxyType` захищає лише верхній рівень і показує зміни вихідного словника, тому перед обгортанням джерело копіюють; для вкладених структур потрібне «глибоке заморожування», яке рекурсивно перетворює `dict`/`set`/`list` на `MappingProxyType`/`frozenset`/`tuple`. Ціна незмінності — створення нового об'єкта на кожне оновлення, але це компенсується відсутністю захисних копій у коді та можливістю кешувати чисті розрахунки.

Хід роботи:

1. За варіантом 13 (страхові поліси) спочатку зафіксував мутабельну модель «до» у `mutable_model.py`: поліс як `dict` зі списком покриттів, множиною ризиків

і вкладеним словником метаданих; усі оновлення — in-place, премія щоразу перераховується. Файл лишено для порівняння часу:

```python
RISK_FACTORS = {"low": 0.9, "medium": 1.0, "high": 1.5}


def make_policy_dict(id: int, holder: str) -> dict[str, Any]:
    """Поліс як звичайний dict зі списком, множиною і вкладеним dict."""
    return {
        "id": id,
        "holder": holder,
        "coverages": [],
        "risks": set(),
        "meta": {},
        "active": True,
    }


def add_coverage_inplace(
    policy: dict[str, Any], name: str, amount: float, rate: float
) -> None:
    """Мутує список покриттів."""
    policy["coverages"].append({"name": name, "amount": amount, "rate": rate})


def add_risk_inplace(policy: dict[str, Any], risk: str) -> None:
    """Мутує множину ризиків."""
    policy["risks"].add(risk)


def final_premium_dict(policy: dict[str, Any]) -> float:
    """Премія без кешу: щоразу перераховується."""
    base = sum(c["amount"] * c["rate"] for c in policy["coverages"])
    factor = 1.0
    for r in policy["risks"]:
        factor *= RISK_FACTORS[r]
    return float(base * factor)
```

2. У `model.py` описав незмінну модель: тип ризику через `Literal`, сутності `Coverage` і `Policy` як `@dataclass(frozen=True, slots=True)`. Покриття зберігаються у `tuple`, ризики у `frozenset`, метадані клієнта — як `Mapping` (read-

only proxy). Методи `add_coverage`, `add_risk`, `deactivate` повертають нові копії через `replace`, конкатенацію tuple та об'єднання frozenset:

```python
Risk = Literal["low", "medium", "high"]


@dataclass(frozen=True, slots=True)
class Coverage:
    """Одне покриття полісу: назва, страхова сума і ставка премії."""

    name: str
    amount: float
    rate: float

    def premium(self) -> float:
        """Премія за це покриття."""
        return self.amount * self.rate


@dataclass(frozen=True, slots=True)
class Policy:
    """Страховий поліс: покриття (tuple), ризики (frozenset), meta (read-only)."""

    id: int
    holder: str
    coverages: tuple[Coverage, ...]
    risks: frozenset[Risk]
    # mappingproxy не хешується → виключаємо з hash (у eq лишається)
    meta: Mapping[str, str] = field(hash=False)
    active: bool = True

    def base_premium(self) -> float:
        """Сума премій усіх покриттів без урахування ризиків."""
        return sum(c.premium() for c in self.coverages)

    def add_coverage(self, coverage: Coverage) -> Policy:
        """Новий поліс із доданим покриттям."""
        return replace(self, coverages=self.coverages + (coverage,))

    def add_risk(self, risk: Risk) -> Policy:
        """Новий поліс із доданим ризиком."""
        return replace(self, risks=self.risks | frozenset({risk}))

    def deactivate(self) -> Policy:
        """Новий поліс зі знятою активністю."""
        return replace(self, active=False)

```

3. Додав `make_meta`, яка обгортає у `MappingProxyType` копію словника (щоб ніхто зовні не тримав посилання на джерело), і зручний конструктор `make_policy`:

```python
def make_meta(**kwargs: str) -> Mapping[str, str]:
    """Read-only view над КОПІЄЮ словника, щоб ніхто не тримав посилання на оригінал."""
    return MappingProxyType(dict(kwargs))


def make_policy(
    id: int,
    holder: str,
    coverages: tuple[Coverage, ...] = (),
    risks: frozenset[Risk] = frozenset(),
    **meta: str,
) -> Policy:
    """Зручний конструктор полісу з read-only meta."""
    return Policy(id, holder, coverages, risks, make_meta(**meta))
```

4. У `core.py` реалізував чисті бізнес-функції: таблицю коефіцієнтів ризику як read-only mapping, `risk_factor`, кешований через `@lru_cache` розрахунок

`final_premium` (можливий лише тому, що `Policy` хешований) та pure-оновлення `add_coverage`, `add_risk`, `with_meta`, `recalculate`:

```python
from model import Coverage, Policy, Risk

RISK_FACTORS: Mapping[Risk, float] = MappingProxyType(
    {"low": 0.9, "medium": 1.0, "high": 1.5}
)


def risk_factor(risks: frozenset[Risk]) -> float:
    """Добуток коефіцієнтів усіх ризиків полісу (порожня множина → 1.0)."""
    factor = 1.0
    for r in risks:
        factor *= RISK_FACTORS[r]
    return factor


@lru_cache(maxsize=1024)
def final_premium(policy: Policy) -> float:
    """Кінцева премія = базова премія × добуток коефіцієнтів ризику. Кешується."""
    return policy.base_premium() * risk_factor(policy.risks)


def add_coverage(policy: Policy, name: str, amount: float, rate: float) -> Policy:
    """Pure update: новий поліс із доданим покриттям."""
    return policy.add_coverage(Coverage(name, amount, rate))


def add_risk(policy: Policy, risk: Risk) -> Policy:
    """Pure update: новий поліс із доданим ризиком."""
    return policy.add_risk(risk)


def with_meta(policy: Policy, **updates: str) -> Policy:
    """Pure update: новий поліс із новою read-only meta (копія + оновлення)."""
    merged = {**policy.meta, **updates}
    return Policy(
        policy.id,
        policy.holder,
        policy.coverages,
        policy.risks,
        MappingProxyType(merged),
        policy.active,
    )


def recalculate(policy: Policy) -> tuple[Policy, float]:
    """Повертає той самий поліс і його кінцеву премію (кешовану)."""
    return policy, final_premium(policy)
```

5. Додав `index_by_policy`, де поліс використовується як ключ словника, та утиліту `freeze` для глибокого заморожування вкладених структур. Поле `meta` (`mappingproxy`) не має `__hash__`, тому в моделі воно виключене з хешу через `field(hash=False)`, але бере участь у порівнянні:

```python
def index_by_policy(policies: tuple[Policy, ...]) -> Mapping[Policy, float]:
    """Поліс як ключ словника (хешованість frozen dataclass) → премія."""
    return MappingProxyType({p: final_premium(p) for p in policies})


def freeze(obj: Any) -> Any:
    """Глибоке заморожування: dict → MappingProxyType, set → frozenset, list → tuple."""
    if isinstance(obj, Mapping):
        return MappingProxyType({k: freeze(v) for k, v in obj.items()})
    if isinstance(obj, (set, frozenset)):
        return frozenset(freeze(x) for x in obj)
    if isinstance(obj, (list, tuple)):
        return tuple(freeze(x) for x in obj)
    return obj
```

6. Демонстраційний скрипт `main.py` будує ланцюжок із трьох pure-оновлень, показує, що початковий поліс не змінився, друкує стан кешу та перевіряє read-only поведінку метаданих:

```python
def main() -> None:
    """Створює поліс, виконує три pure-оновлення і показує, що оригінал не змінився."""
    p0 = make_policy(
        1,
        "Ivan Petrenko",
        (Coverage("car", 10_000.0, 0.05),),
        frozenset({"low"}),
        source="web",
    )
    p1 = add_coverage(p0, "home", 20_000.0, 0.01)
    p2 = add_risk(p1, "high")
    p3 = with_meta(p2, agent="A-7")

    for name, p in [("p0", p0), ("p1", p1), ("p2", p2), ("p3", p3)]:
        print(
            f"{name}: coverages={len(p.coverages)} risks={sorted(p.risks)} "
            f"meta={dict(p.meta)} base={p.base_premium():.2f} "
            f"final={final_premium(p):.2f}"
        )
    print(
        "p0 is p3:",
        p0 is p3,
        "| p0 unchanged:",
        p0.coverages == (Coverage("car", 10_000.0, 0.05),),
    )
    print("cache:", final_premium.cache_info())
    index = index_by_policy((p0, p2))
    print(
        "as dict keys:",
        len(index),
        "policies ->",
        [round(v, 2) for v in index.values()],
    )
    try:
        p0.meta["source"] = "x"  # type: ignore[index]
    except TypeError as e:
        print("meta read-only:", e)


if __name__ == "__main__":
    main()
```

7. Написав тести у `tests/test_immutability.py`: відсутність мутації після додавання покриття та ризику, `FrozenInstanceError` при присвоєнні поля,

`TypeError` при записі у proxy, копіювання джерела `make_meta`, нова мапа у `with_meta`:

```python
def test_add_coverage_returns_new_policy(policy: Policy) -> None:
    before = (policy.coverages, policy.risks, dict(policy.meta))
    new = add_coverage(policy, "home", 20_000.0, 0.01)
    assert (policy.coverages, policy.risks, dict(policy.meta)) == before
    assert new is not policy
    assert len(new.coverages) == 3
    assert new.base_premium() == pytest.approx(policy.base_premium() + 200.0)


def test_add_risk_returns_new_policy(policy: Policy) -> None:
    new = add_risk(policy, "high")
    assert policy.risks == frozenset({"low"})
    assert new.risks == frozenset({"low", "high"})
    assert new.deactivate().active is False and policy.active is True


def test_frozen_fields_cannot_be_assigned(policy: Policy) -> None:
    with pytest.raises(FrozenInstanceError):
        policy.holder = "x"  # type: ignore[misc]


def test_meta_is_read_only(policy: Policy) -> None:
    with pytest.raises(TypeError):
        policy.meta["source"] = "changed"  # type: ignore[index]
    assert policy.meta["source"] == "web"


def test_make_meta_copies_source() -> None:
    raw = {"k": "v"}
    meta = make_meta(**raw)
    raw["k"] = "changed"
    assert meta["k"] == "v"


def test_with_meta_creates_new_mapping(policy: Policy) -> None:
    new = with_meta(policy, agent="B-1", channel="phone")
    assert dict(new.meta) == {"source": "web", "agent": "B-1", "channel": "phone"}
    assert dict(policy.meta) == {"source": "web", "agent": "A-7"}

```

8. Додав тести на хешованість (поліси у `set` і як ключі словника), на кеш `final_premium` через `cache_info` та на глибоке заморожування:

```python
def test_hashability_and_dict_key(policy: Policy) -> None:
    other = add_risk(policy, "high")
    assert len({policy, other, add_risk(policy, "high")}) == 2
    index = index_by_policy((policy, other))
    assert index[policy] == pytest.approx(540.0)
    assert index[other] == pytest.approx(600.0 * 0.9 * 1.5)
    assert set(index) == {policy, other}


def test_final_premium_and_cache(policy: Policy) -> None:
    final_premium.cache_clear()
    assert final_premium(policy) == pytest.approx((500.0 + 100.0) * 0.9)
    final_premium(policy)
    info = final_premium.cache_info()
    assert info.hits == 1 and info.misses == 1
    assert final_premium(add_risk(policy, "high")) == pytest.approx(600.0 * 0.9 * 1.5)


def test_freeze_deep() -> None:
    frozen = freeze({"a": [1, {2, 3}], "b": {"c": [4]}})
    with pytest.raises(TypeError):
        frozen["a"] = 0
    assert frozen["a"] == (1, frozenset({2, 3}))
    with pytest.raises(TypeError):
        frozen["b"]["c"] = 0
    assert frozen["b"]["c"] == (4,)

```

9. Property-тести на Hypothesis генерують випадкові поліси й покриття та перевіряють, що `add_coverage` чистий, а рівні поліси мають однаковий хеш і однакову кешовану премію:

```python
coverages_st = st.tuples(
    st.text(min_size=1, max_size=8),
    st.floats(min_value=0, max_value=1e6, allow_nan=False),
    st.floats(min_value=0, max_value=1, allow_nan=False),
).map(lambda t: Coverage(*t))
policies_st = st.builds(
    make_policy,
    st.integers(min_value=1),
    st.text(min_size=1, max_size=10),
    st.tuples(coverages_st, coverages_st).map(tuple),
    st.frozensets(st.sampled_from(["low", "medium", "high"])),
)


@given(policies_st, coverages_st)
def test_property_add_coverage_pure(p: Policy, c: Coverage) -> None:
    before = (p.coverages, p.risks, dict(p.meta))
    new = p.add_coverage(c)
    assert (p.coverages, p.risks, dict(p.meta)) == before
    assert new.coverages == p.coverages + (c,)
    assert new.base_premium() == pytest.approx(p.base_premium() + c.premium())


@given(policies_st)
def test_property_hash_consistent_with_eq(p: Policy) -> None:
    twin = Policy(p.id, p.holder, p.coverages, p.risks, p.meta, p.active)
    assert twin == p and hash(twin) == hash(p)
    assert final_premium(p) == final_premium(twin)
```

10. Для вимірювання ефекту у `tests/test_performance.py` два тести з однаковим навантаженням (2000 додавань покриття і 2000 розрахунків премії) на мутабельній моделі «до» та незмінній «після»:

```python
N = 2_000


def test_mutable_before() -> None:
    p = make_policy_dict(1, "x")
    for i in range(N):
        add_coverage_inplace(p, f"c{i}", 100.0, 0.01)
    add_risk_inplace(p, "high")
    total = sum(final_premium_dict(p) for _ in range(N))
    assert total > 0


def test_immutable_after(policy: Policy) -> None:
    p = policy
    for i in range(N):
        p = add_coverage(p, f"c{i}", 100.0, 0.01)
    p = add_risk(p, "high")
    final_premium.cache_clear()
    total = sum(final_premium(p) for _ in range(N))
    assert total > 0
    assert final_premium.cache_info().hits == N - 1
```

11. Запустив демонстрацію та порівняння часу `pytest --durations=5`: незмінна модель із кешованим розрахунком виконує те саме навантаження приблизно у

20 разів швидше (0.01 с проти 0.23 с), бо 1999 із 2000 звернень до премії — cache hit:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab03> python main.py
p0: coverages=1 risks=['low'] meta={'source': 'web'} base=500.00 final=450.00
p1: coverages=2 risks=['low'] meta={'source': 'web'} base=700.00 final=630.00
p2: coverages=2 risks=['high', 'low'] meta={'source': 'web'} base=700.00 final=945.00
p3: coverages=2 risks=['high', 'low'] meta={'source': 'web', 'agent': 'A-7'} base=700.00 final=945.00
p0 is p3: False | p0 unchanged: True
cache: CacheInfo(hits=0, misses=4, maxsize=1024, currsize=4)
as dict keys: 2 policies -> [450.0, 945.0]
meta read-only: 'mappingproxy' object does not support item assignment
```

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab03> python -m pytest -q --durations=5
.............                                                            [100%]
============================= slowest 5 durations =============================
0.98s call     lab03/tests/test_immutability.py::test_property_add_coverage_pure
0.22s call     lab03/tests/test_performance.py::test_mutable_before
0.08s call     lab03/tests/test_immutability.py::test_property_hash_consistent_with_eq
0.01s call     lab03/tests/test_performance.py::test_immutable_after

(1 durations < 0.005s hidden.  Use -vv to show these durations.)
13 passed in 1.56s
```

12. Перевірив проєкт автоматичними інструментами: усі 13 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають зауважень до стилю:

```
C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab03> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs\FunctionalProgramming
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 13 items

tests\test_immutability.py::test_add_coverage_returns_new_policy PASSED  [  7%]
tests\test_immutability.py::test_add_risk_returns_new_policy PASSED      [ 15%]
tests\test_immutability.py::test_frozen_fields_cannot_be_assigned PASSED [ 23%]
tests\test_immutability.py::test_meta_is_read_only PASSED                [ 30%]
tests\test_immutability.py::test_make_meta_copies_source PASSED          [ 38%]
tests\test_immutability.py::test_with_meta_creates_new_mapping PASSED    [ 46%]
tests\test_immutability.py::test_hashability_and_dict_key PASSED         [ 53%]
tests\test_immutability.py::test_final_premium_and_cache PASSED          [ 61%]
tests\test_immutability.py::test_freeze_deep PASSED                      [ 69%]
tests\test_immutability.py::test_property_add_coverage_pure PASSED       [ 76%]
tests\test_immutability.py::test_property_hash_consistent_with_eq PASSED [ 84%]
tests\test_performance.py::test_mutable_before PASSED                    [ 92%]
tests\test_performance.py::test_immutable_after PASSED                   [100%]

============================= 13 passed in 0.67s ==============================

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab03> python -m mypy .
Success: no issues found in 7 source files

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab03> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs\FunctionalProgramming\lab03> python -m black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я перевів мутабельну модель страхових полісів (варіант 13) на незмінну: сутності `Coverage` і `Policy` як frozen dataclass зі slots, покриття у `tuple`, ризики у `frozenset`, метадані клієнта у `MappingProxyType` над копією джерела. Усі оновлення реалізовано як чисті

функції, що повертають нові копії через `dataclasses.replace`; хешованість полісу дозволила кешувати розрахунок премії через `lru_cache` і використовувати поліс як ключ словника. Тести підтвердили відсутність мутацій і read-only поведінку, а порівняння часу показало, що незмінна модель не лише не сповільнила код, а завдяки кешуванню значно його прискорила; накладні витрати — копіювання колекцій при кожному оновленні та захист лише верхнього рівня у proxy.